

Compte rendu — Benchmark CPU vs GPU sur opérations vectorielles

Objectif

Comparer les performances **CPU vs GPU** (PyTorch) sur trois opérations vectorielles — **somme**, **produit scalaire**, **norme** — pour des vecteurs allant de **1 000 à 100 millions** d'éléments, et identifier le "point de bascule" à partir duquel le GPU devient plus rapide que le CPU.

Méthodologie

- **Portabilité** : détection automatique du matériel (CUDA, MPS Apple Silicon, ou CPU seul) — le même notebook tourne sur Colab, PC NVIDIA ou Mac.
- **Rigueur de mesure** : trois bonnes pratiques appliquées correctement
 - *warm-up* (3 exécutions ignorées avant mesure)
 - *synchronisation* explicite (`cuda.synchronize()` / `mps.synchronize()`) pour ne pas mesurer uniquement le lancement asynchrone du kernel
 - *médiane* sur 10 répétitions, plus robuste que la moyenne aux perturbations système
- Les tenseurs sont créés **directement sur le device cible** avant chronométrage, pour isoler le temps de calcul du temps de transfert mémoire.
- 6 tailles testées (1K → 100M), 3 opérations, 2 devices → 36 mesures.

Environnement d'exécution

GPU détecté : **Tesla T4** (15,64 Go), PyTorch 2.11.0+cu128

Taille	Somme	Produit scalaire	Norme
1 000	0,16×	0,10×	0,19×
100 000	0,20×	0,37×	0,61×
1 000 000	4,73×	6,02×	5,28×
10 000 000	13,26×	15,56×	18,13×
100 000 000	22,31×	19,20×	22,58×

Point de bascule identifié empiriquement : entre 100 000 et 1 000 000 éléments, le GPU passe devant pour les trois opérations.

Analyse

- **Petits vecteurs ($\leq 100K$)** : le CPU gagne nettement (jusqu'à 10× plus rapide), à cause du coût fixe de lancement d'un kernel GPU (latence d'appel non compensée par le peu de calcul à faire).
- **Grands vecteurs ($\geq 10M$)** : le GPU l'emporte largement, avec des speedups de 13× à 23×, cohérent avec le parallélisme massif du GPU sur des opérations de réduction.
- Le **produit scalaire** (qui lit 2 vecteurs) montre l'avantage GPU le plus précoce (6× dès 1M), confirmant l'intuition posée dans le notebook.